# Table of Contents


Universitatea Politehnica Timișoara ..... 2
Facultatea de Automatică și Calculatoare ..... 2
Proiectarea cu Microprocesoare ..... 2
Implementarea unui microsistem cu microprocesorul 8086 ..... 2
Tema proiectului ..... 2
Descrierea hardware-ului ..... 3
1. Unitatea centrală ..... 3
2. Memoriile ..... 3
3. Interfețele serială și paralelă ..... 4
4. Perifericele ..... 6
Descrierea software ..... 6
Definiții (constante) ..... 6
1 și 2. Rutinele de programare 8251 și 8255 ..... 7
3 și 4. Emisie / recepție caracter pe interfața serială ..... 7
5. Emisie caracter pe interfața paralelă ..... 8
6. Scanarea minitastaturii ..... 8
7. Aprindere / stingere LED ..... 9
8. Afișare caracter hexazecimal pe un rang ..... 9
Bibliografie ..... 10
Anexe — Schemele microsistemului ..... 10
Anexa 1 — Unitatea centrală ..... 11
Anexa 2 — Memoriile ..... 12
Anexa 3 — Interfețele serială și paralelă ..... 13
Anexa 4 — Perifericele (afișaj, LED-uri, tastatură) ..... 14

Universitatea Politehnica Timișoara

Facultatea de Automatică și Calculatoare

# Proiectarea cu Microprocesoare

## Implementarea unui microsistem cu microprocesorul 8086

**Autor:** Bordea Sebastian-Florin **Grupa / Subgrupa:** TI 1.1 **Anul universitar:** 2025–2026

## Tema proiectului

Să se proiecteze un microsistem cu următoarea structură:

- unitate centrală cu microprocesorul 8086;
- 128 KB memorie EPROM, utilizând circuite 27C2048;
- 64 KB memorie SRAM, utilizând circuite 62512;
- interfață serială, cu circuitul 8251, plasată în zona 0AF0H – 0AF2H sau 0BF0H – 0BF2H, în funcție de poziția microcomutatorului S1;
- interfață paralelă, cu circuitul 8255, plasată în zona 0D70H – 0D76H sau 0C70H – 0C76H, în funcție de poziția microcomutatorului S2;
- o minitastatură cu 12 contacte;
- 6 led-uri;
- un modul de afișare cu 7 segmente, cu 6 ranguri.

Toate programele în limbaj de asamblare sunt concepute sub formă de subrutine: rutinele de programare ale circuitelor 8251 și 8255, rutinele de emisie/recepție caracter pe interfața serială, rutina de emisie caracter pe interfața paralelă, rutina de scanare a minitastaturii, rutina de aprindere/stingere a unui led și rutina de afișare a unui caracter hexazecimal pe un rang cu segmente.

**Opțiuni alese în proiect:** - Microcomutator **S1** → interfața serială la **0BF0H – 0BF2H**. - Microcomutator **S2** → interfața paralelă la **0C70H – 0C76H**.

## Descrierea hardware-ului

Microsistemul este organizat pe patru planșe funcționale: unitatea centrală, memoriile, interfețele serială și paralelă, și perifericele. Schemele complete sunt atașate ca anexe.

### 1. Unitatea centrală

Nucleul sistemului este microprocesorul **Intel 8086**, un procesor pe 16 biți, alimentat la +5V, cu magistrală de date pe 16 biți și magistrală de adrese pe 20 de biți (adresare directă a 1 MB de memorie). Procesorul este configurat în **mod minim** (pinul MN/MX# conectat la +5V), caz în care generează el însuși semnalele de control: ALE, RD#, WR#, M/IO#, DT/R#, DEN#.

Componentele auxiliare ale unității centrale:

- **Generatorul de tact 8284A** — primește semnalul de la un cuarț și furnizează tactul stabil CLK pentru procesor, PCLK pentru periferice, precum și semnalele de sincronizare RESET și READY.
- **Latch-uri de adrese 74LS373 (×3)** — magistrala AD0–AD15 este multiplexată (adrese și date pe aceleași linii). Semnalul ALE comandă latch-urile care „prind” adresa la începutul ciclului:
  - U3: AD0–AD7 → A0–A7
  - U4: AD8–AD15 → A8–A15
  - U5: A16–A19 și BHE# → adrese demultiplexate
- **Transceivere de date 74LS245 (×2)** — separă și amplifică magistrala de date bidirecțională. Pinul DIR este comandat de DT/R#, iar OE# de DEN#:
  - U6: date D0–D7
  - U7: date D8–D15

### 2. Memoriile

Spațiul de adresare al lui 8086 este de 1 MB (00000H–FFFFFH). Harta de memorie:

| Zonă | Circuit | Dimensiune | Interval adrese |
|---|---|---|---|
| SRAM | 62512 (echiv. 62256 ×2) | 64 KB | 00000H – 0FFFFH |
| EPROM | 27C2048 | 128 KB | E0000H – FFFFFH |

- **EPROM 27C2048** — memorie nevolatilă pentru cod, plasată la finalul spațiului de adrese (lângă FFFF0H, vectorul de reset al lui 8086). Circuitul este organizat pe 16 biți (128K × 16), deci se conectează direct pe toată magistrala de date D0–D15, cu un singur circuit. Pinii: A0–A16 (adrese), O0–O15 (date), CE# (chip enable), OE# (legat la RD#).

- **SRAM 62256 ×2** — memorie volatilă pentru date și stivă, plasată la începutul spațiului. Organizarea pe bancuri par/impar (câte un circuit de 32K×8 pentru octeții pari D0–D7 și impari D8–D15) acoperă cei 64 KB pe magistrala de 16 biți. Scrierea independentă pe octet se face prin semnalele ~WE_PAR (din WR# și A0) și ~WE_IMPAR (din WR# și BHE#), generate cu porți SAU (74LS32).

  Notă: Tema cere circuitul SRAM **62512**, care nu este disponibil în biblioteca EasyEDA. S-a folosit echivalentul funcțional **62256** (SRAM 32K×8), câte două circuite, pentru a acoperi cei 64 KB.

**Decodificarea memoriilor** se face cu un **74LS138** (decodor 3-la-8) care primește biții de adresă A17, A18, A19 și este validat de M/IO#.

Tabelul de decodificare completă (se reține biții A19, A18, A17 care rămân constanți pe tot intervalul fiecărui bloc de memorie):

| Mem. | A19 | A18 | A17 | A16 | A15 | A14 | A13 | A12 | A11 | A10 | A9 | A8 | A7 | A6 | A5 | A4 | A3 | A2 | A1 | A0 | Adresă |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SRAM | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 00000H |
| SRAM | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0FFFFH |
| EPROM | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | E0000H |
| EPROM | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | FFFFFH |

Biții A19, A18, A17 (îngroșați conceptual) sunt cei care rămân constanți și determină selecția blocului. Funcțiile de selecție rezultate:

```
SEL_SRAM  = ~A19 & ~A18 & ~A17  =  ~Y0   (A19 A18 A17 = 0 0 0)  ->  00000H
SEL_EPROM =  A19 &  A18 &  A17  =  ~Y7   (A19 A18 A17 = 1 1 1)  ->  E0000H
```

## 3. Interfețele serială și paralelă

- **Interfața serială 8251 (USART)** — convertește datele paralele în flux serial pentru transmisie (și invers la recepție). Este programată prin pinul C/D# (legat la A1): la C/D#=0 se accesează registrul de date, la C/D#=1 registrul de

comandă/stare. Semnalul serial de nivel TTL este convertit la nivel RS-232 prin circuitul **MAX232**. Adresa de bază: **0BF0H** (S1).

- **Interfața paralelă 8255 (PPI)** — oferă trei porturi de 8 biți (A, B, C), folosite în **mod 0**. Adresa de bază: **0C70H** (S2). Alocarea porturilor:
  - Port A (ieșire) → segmentele afișajului 7 segmente
  - Port B (intrare) → coloanele minitastaturii
  - Port C (ieșire) → PC0–PC2 selecție rang/linie, PC3–PC7 LED-uri

**Decodificarea porturilor I/O** se face cu un al doilea **74LS138**, care primește biții A8, A9, A11 și, prin microcomutatoarele S1/S2, selectează interfața activă. Comutatorul SPDT alege care ieșire a decodorului ajunge la pinul CS# al circuitului (poziția 1 sau poziția 2), permițând mutarea ferestrei de adrese.

Tabelul de decodificare completă a porturilor (biții A11, A9, A8 determină ieșirea decodorului):

| Adresă | A15 | A14 | A13 | A12 | A11 | A10 | A9 | A8 | A7 | A6 | A5 | A4 | A3 | A2 | A1 | A0 | Interfață |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0AF0H | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 8251 date, S1=0 |
| 0BF0H | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 8251 date, S1=1 |
| 0C70H | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 8255 PA, S2=0 |
| 0D70H | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 8255 PA, S2=1 |

Biții A11, A9, A8 sunt cei de selecție (intrările decodorului). Funcțiile:

```
SEL_8251 (S1=1) = ~A11 & ~A9 &  A8  =  ~Y1   ->  0BF0H
SEL_8255 (S2=0) = ~A11 &  A9 & ~A8  =  ~Y2   ->  0C70H
```

În plus, în zona serială, bitul A1 (legat la C/D#) selectează între registrul de date (A1=0, adresa 0BF0H) și registrul de control/stare (A1=1, adresa 0BF2H). În zona paralelă, biții A1 și A2 selectează între Port A (0C70H), Port B (0C72H), Port C (0C74H) și registrul de control (0C76H).

### 4. Perifericele

- **Afișaj 7 segmente, 6 ranguri** — multiplexat. Segmentele (a–g, dp) sunt comune tuturor rangurilor și sunt comandate de Port A. Selecția rangului activ se face prin PC0–PC2, decodate de un 74LS138 care activează anodul comun al unui singur rang. Procesorul baleiază rapid cele 6 ranguri, astfel încât toate cifrele apar aprinse simultan.
- **6 LED-uri** — conectate pe Port C (PC3–PC7) prin rezistoare de limitare de 330 Ω. Starea LED-urilor este menținută într-o variabilă-oglindă software, deoarece 8255 nu permite citirea sigură a unui port de ieșire.
- **Minitastatură 12 taste** — matrice 3 linii × 4 coloane. Liniile sunt comandate prin PC0–PC2, coloanele sunt citite prin Port B (PB0–PB3), cu rezistoare de pull-up de 10 kΩ. Apăsarea unei taste se detectează activând pe rând o linie și citind coloanele.

## Descrierea software

Toate programele sunt scrise ca subrutine (`proc`/`endp`), în limbaj de asamblare x86 (8086, mod real), în sintaxă MASM/TASM. Perifericele se accesează prin porturi de intrare/ieșire (instrucțiunile `IN` și `OUT`, prin registrul DX), nu prin întreruperi DOS.

### Definiții (constante)

```
; --- INTERFATA SERIALA (8251), zona 0BF0h-0BF2h (microcomutator S1) ---
SER_DATA    EQU 0BF0h        ; registru de date (RX / TX)
SER_CTRL    EQU 0BF1h        ; registru control (scriere) / status (citir
e)
SER_TXRDY   EQU 00000001b    ; status bit 0: emitator gata
SER_RXRDY   EQU 00000010b    ; status bit 1: octet receptionat
SER_MOD     EQU 01001110b    ; 4Eh: asincron, 8 date, fara paritate, 1 st
op, x16
SER_CMD     EQU 00110111b    ; 37h: TxEN, DTR, RxEN, ErrReset, RTS

; --- INTERFATA PARALELA (8255), zona 0C70h-0C76h (microcomutator S2) ---
PAR_PORTA   EQU 0C70h        ; Port A
PAR_PORTB   EQU 0C72h        ; Port B
PAR_PORTC   EQU 0C74h        ; Port C
PAR_CTRL    EQU 0C76h        ; registru de Control
PAR_CFG     EQU 10000010b    ; 82h: A iesire, B intrare, C iesire (mod 0)
```

Cuvântul de control al lui 8255 (82H = 10000010b) înseamnă: bit7=1 (mod I/O activ), Port A ieșire, Port B intrare, Port C ieșire, toate în mod 0.

## 1 și 2. Rutinele de programare 8251 și 8255

```
; init_8251 - programarea interfetei seriale
; Dupa reset, 8251 asteapta intai cuvantul de MOD, apoi COMANDA.
init_8251 proc
    MOV DX, SER_CTRL
    MOV AL, SER_MOD      ; cuvant de mod: asincron 8N1, factor x16
    OUT DX, AL
    MOV AL, SER_CMD      ; cuvant de comanda: TxEN, RxEN, DTR, RTS, ErrRe
set
    OUT DX, AL
    RET
init_8251 endp

; init_8255 - programarea interfetei paralele
init_8255 proc
    MOV DX, PAR_CTRL
    MOV AL, PAR_CFG      ; A iesire, B intrare, C iesire (mod 0)
    OUT DX, AL
    RET
init_8255 endp
```

## 3 și 4. Emisie / recepție caracter pe interfața serială

```
; ser_emisie - INTRARE: AL = octetul de transmis
; Asteapta TxRDY=1 in status, apoi scrie octetul in registrul de date.
ser_emisie proc
    PUSH AX              ; salvam octetul cat testam status-ul
asteapta_tx:
    MOV DX, SER_CTRL
    IN  AL, DX           ; citeste registrul de status
    TEST AL, SER_TXRDY   ; emitatorul e gata?
    JZ  asteapta_tx
    POP AX
    MOV DX, SER_DATA
    OUT DX, AL           ; scrie octetul pe linia seriala
    RET
ser_emisie endp

; ser_receptie - IESIRE: AL = octetul receptionat
; Asteapta RxRDY=1 in status, apoi citeste registrul de date.
ser_receptie proc
asteapta_rx:
    MOV DX, SER_CTRL
    IN  AL, DX
    TEST AL, SER_RXRDY   ; a sosit un octet?
    JZ  asteapta_rx
```

```
    MOV DX, SER_DATA
    IN  AL, DX          ; citeste octetul receptionat
    RET
ser_receptie endp
```

## 5. Emisie caracter pe interfața paralelă

```
; par_emisie - INTRARE: AL = octetul de scos pe Port A
par_emisie proc
    MOV DX, PAR_PORTA
    OUT DX, AL
    RET
par_emisie endp
```

## 6. Scanarea minitastaturii

```
; scan_tastatura - IESIRE: AL = codul tastei (0..11) sau 0FFh daca niciun
a
; Matrice 3 linii x 4 coloane. Activeaza pe rand o linie pe PC0..PC2,
; citeste coloanele pe Port B, calculeaza cod = linie * 4 + coloana.
scan_tastatura proc
    XOR BH, BH          ; BH = numarul liniei curente (0..2)
linie_urm:
    CMP BH, NR_LINII
    JAE nicio
    MOV AL, portc_stare
    AND AL, NOT RANG_MASK ; sterge PC0..PC2 (pastreaza LED-urile)
    OR  AL, BH          ; pune codul liniei pe PC0..PC2
    MOV portc_stare, AL
    MOV DX, PAR_PORTC
    OUT DX, AL
    MOV DX, PAR_PORTB
    IN  AL, DX          ; citeste coloanele
    XOR BL, BL
coloana_urm:
    CMP BL, NR_COLOANE
    JAE linie_gata
    MOV CL, BL
    MOV AH, 1
    SHL AH, CL          ; masca bitului coloanei
    TEST AL, AH
    JNZ gasit
    INC BL
    JMP coloana_urm
linie_gata:
    INC BH
    JMP linie_urm
gasit:
```

```
    MOV AL, BH
    MOV CL, NR_COLOANE
    MUL CL                ; AX = linie * 4
    ADD AL, BL            ; AL = cod tasta
    RET
nicio:
    MOV AL, NICIO_TASTA
    RET
scan_tastatura endp
```

## 7. Aprindere / stingere LED

```
; led - INTRARE: BL = numarul LED-ului (0..5), CL = actiune (0 stinge / 1
 aprinde)
; LED-urile sunt pe Port C de la bitul LED_BAZA (PC3). Se modifica doar b
itul
; LED in oglinda portc_stare, fara a atinge bitii de selectie PC0..PC2.
led proc
    MOV AL, 1
    MOV CH, CL            ; pastram actiunea
    MOV CL, BL
    ADD CL, LED_BAZA      ; pozitia reala a bitului
    SHL AL, CL            ; masca bitului LED-ului
    CMP CH, 0
    JZ  led_stinge
led_aprinde:
    OR  portc_stare, AL   ; pune bitul pe 1
    JMP led_scoate
led_stinge:
    NOT AL
    AND portc_stare, AL   ; pune bitul pe 0
led_scoate:
    MOV AL, portc_stare
    MOV DX, PAR_PORTC
    OUT DX, AL
    RET
led endp
```

## 8. Afișare caracter hexazecimal pe un rang

```
; afis_hexa - INTRARE: AL = valoarea (0..0Fh), BL = rangul (0..5)
; Cauta codul in tabela seg7[AL], il scoate pe Port A (segmente), apoi
; selecteaza rangul BL pe PC0..PC2 (via 74LS138), pastrand bitii LED.
seg7 db 3Fh,06h,5Bh,4Fh,66h,6Dh,7Dh,07h   ; 0 1 2 3 4 5 6 7
     db 7Fh,6Fh,77h,7Ch,39h,5Eh,79h,71h   ; 8 9 A b C d E F

afis_hexa proc
    AND AL, 0Fh
```

```
    MOV SI, OFFSET seg7
    XOR AH, AH
    ADD SI, AX          ; SI -> seg7[AL]
    MOV AL, [SI]        ; AL = cod 7 segmente
    MOV DX, PAR_PORTA
    OUT DX, AL          ; scoate segmentele
    MOV AL, portc_stare
    AND AL, NOT RANG_MASK
    AND BL, RANG_MASK
    OR  AL, BL          ; pune codul rangului pe PC0..PC2
    MOV portc_stare, AL
    MOV DX, PAR_PORTC
    OUT DX, AL
    RET
afis_hexa endp
```

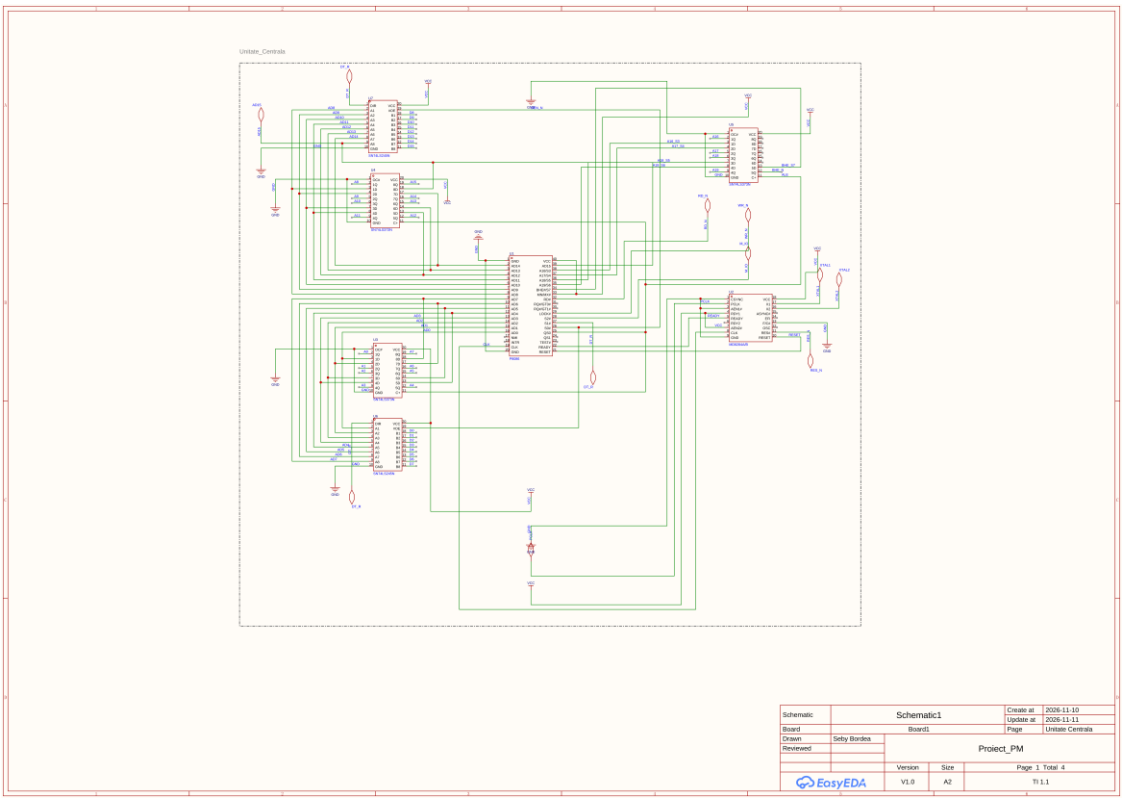
## Bibliografie


1. Mircea Popa, *Sisteme cu microprocesoare*, Editura Orizonturi Universitare, 2003.
2. D.V. Hall, *Microprocessors and Interfacing: Programming and Hardware*, McGraw-Hill, 1992.
3. *Datasheet 27C2048* (EPROM), STMicroelectronics / Microchip.
4. *Datasheet 62256* (SRAM), Cypress / Infineon.
5. *Datasheet 8255A — Programmable Peripheral Interface*, Intel Corporation.
6. *Datasheet 8251A — Programmable Communication Interface*, Intel Corporation.
7. *Datasheet 8086 — 16-bit Microprocessor*, Intel Corporation.
8. Îndrumătorul de laborator și materialele de la disciplina Proiectarea cu Microprocesoare.


## Anexe — Schemele microsistemului

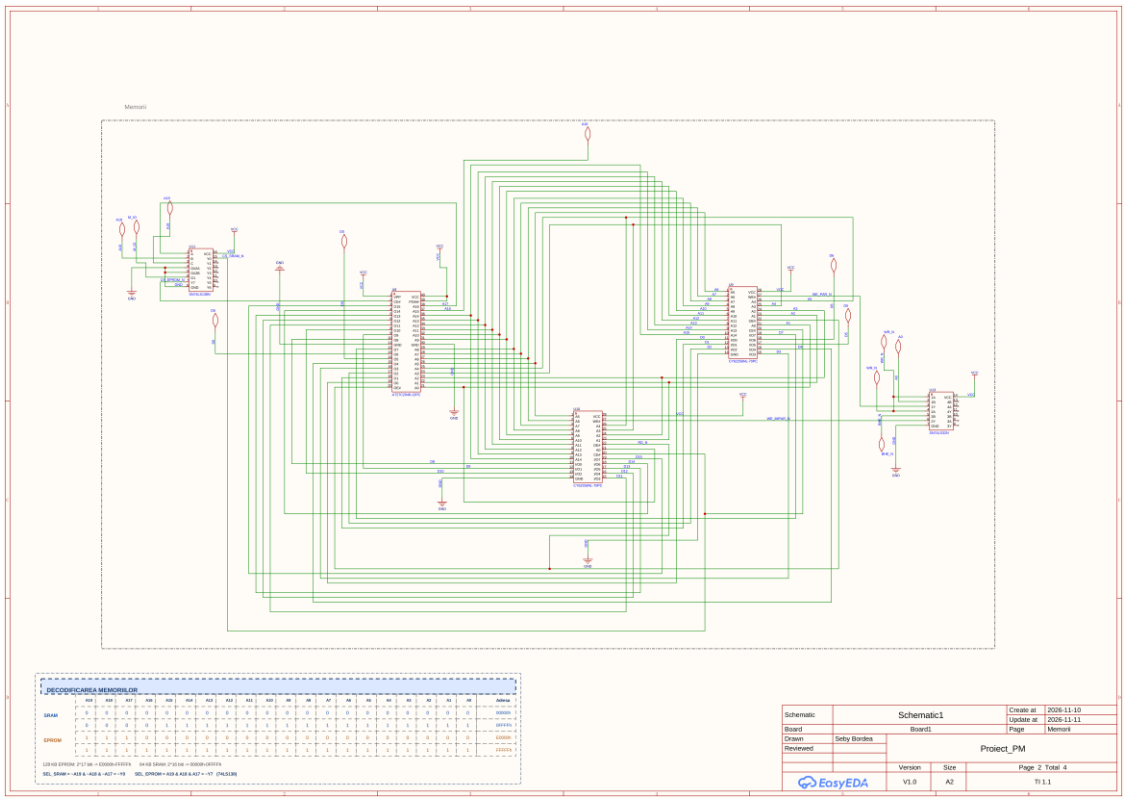
Schemele complete au fost realizate în EasyEDA.

## Anexa 1 — Unitatea centrală



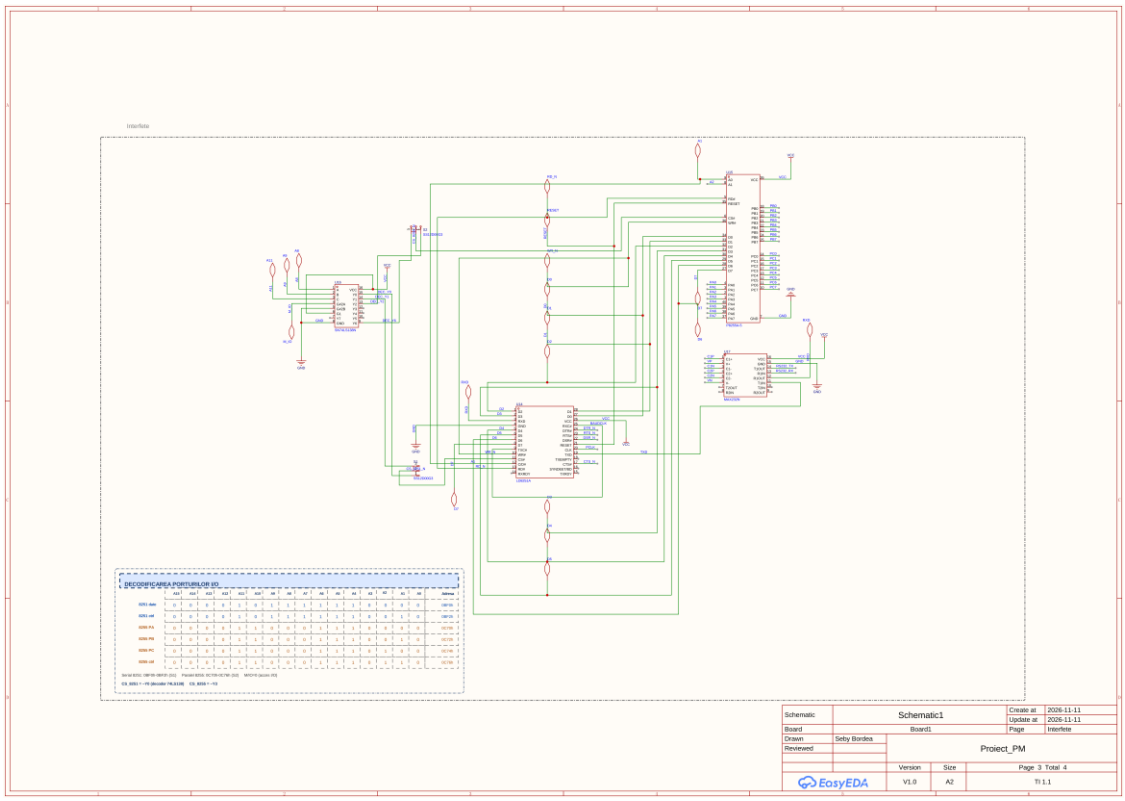

*Schema unitatii centrale*

# Anexa 2 — Memoriile



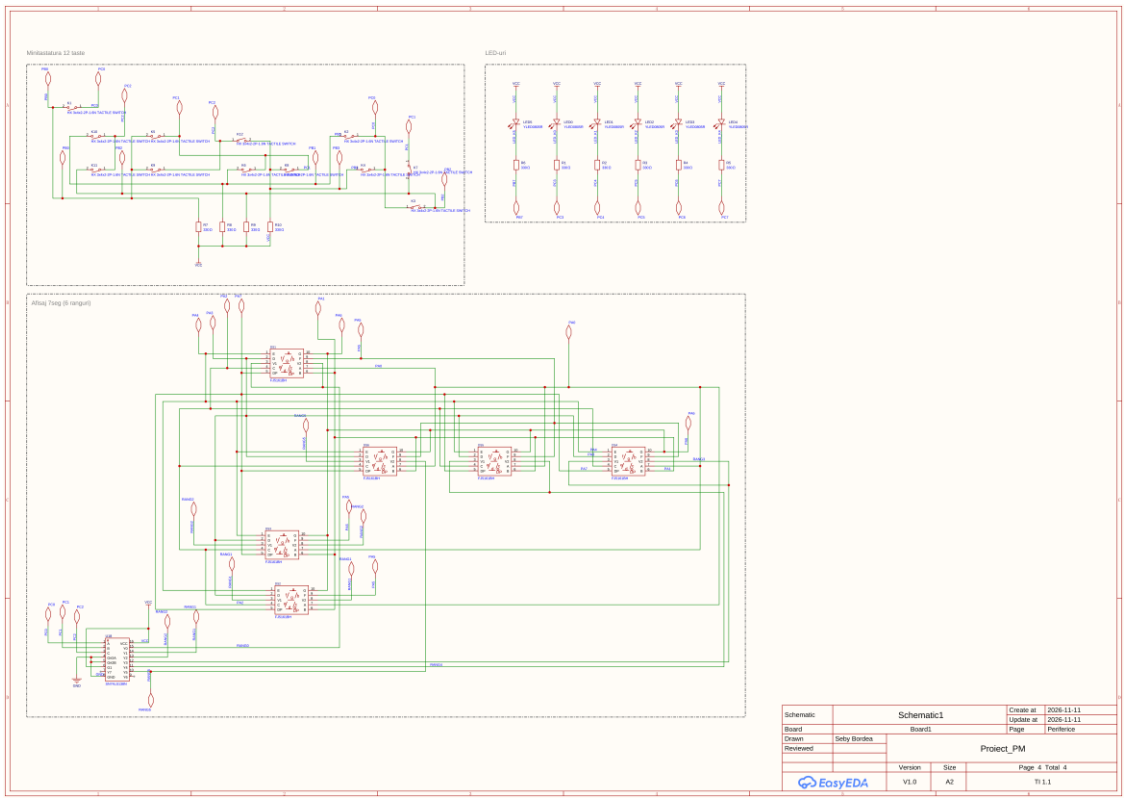

*Schema memoriilor*

## Anexa 3 — Interfețele serială și paralelă




*Schema interfetelor*

## Anexa 4 — Perifericele (afișaj, LED-uri, tastatură)




*Schema perifericelor*